

W09D04 — Layer Caching & Image Optimization

ML Engineer Journey · Phase 3 — Docker & Clean Architecture · 7 May 2026

1. Analogi: memasak yang efisien

Bayangkan kamu memasak nasi goreng setiap pagi. Prosesnya selalu sama: panaskan wajan, tumis bawang, masukkan nasi, tambahkan bumbu, tambahkan kecap. Kalau bumbu berubah, kamu cukup mengulang dari langkah bumbu — tidak perlu memanaskan wajan dari nol karena wajan sudah panas dan bawang sudah matang.

Docker layer caching bekerja persis seperti itu. Setiap instruksi Dockerfile adalah satu langkah memasak. Kalau langkah ke-4 berubah, Docker mengulang dari sana — bukan dari awal. Masalah terjadi ketika developer menaruh COPY . . (menyalin semua kode) di awal Dockerfile. Setiap kali satu baris kode berubah, Docker membatalkan semua cache dari titik itu ke bawah — termasuk RUN pip install yang bisa memakan 2-3 menit.

2. Cara Docker layer caching bekerja

Setiap instruksi RUN, COPY, atau ADD di Dockerfile menghasilkan layer baru yang bersifat read-only. Docker menyimpan layer ini di cache lokal. Saat kamu jalankan docker build lagi, Docker membandingkan tiap instruksi dengan cache yang ada.

Kondisi	Hasil
Instruksi sama persis	Cache HIT — layer diambil dari cache, tidak dieksekusi ulang
Instruksi berubah	Cache MISS — layer dieksekusi ulang
File yang di-COPY berubah	Cache MISS — Docker hash konten file, bukan nama
Layer sebelumnya cache miss	Semua layer sesudahnya otomatis cache MISS (efek domino)

■ Efek domino — poin paling penting

Satu layer cache miss → SEMUA layer sesudahnya ikut diulang dari nol.
Bukan hanya layer yang berubah — seluruh layer di bawahnya ikut terkena.
Ini mengapa urutan instruksi di Dockerfile sangat menentukan kecepatan development loop.

3. Urutan instruksi yang optimal

Aturan emasnya sederhana: letakkan yang jarang berubah di atas, yang sering berubah di bawah. Dependencies (requirements.txt) jarang berubah. Source code (app/) sering berubah setiap development session.

Urutan BURUK ■

Urutan OPTIMAL ■

FROM python:3.11-slim	FROM python:3.11-slim
WORKDIR /app	WORKDIR /app
COPY . . ← masalah di sini	COPY requirements.txt . ← hanya ini dulu
RUN pip install -r requirements.txt	RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8000	COPY . . ← baru seluruh kode
CMD [...]	EXPOSE 8000
	CMD [...]

Dockerfile optimal — penjelasan tiap baris

```
FROM python:3.11-slim
WORKDIR /app

# Salin HANYA requirements.txt dulu — bukan seluruh kode
# Docker cache layer ini selama requirements.txt tidak berubah
COPY requirements.txt .

# Layer paling mahal (~390 MB) — dijaga agar selalu di-cache
# --no-cache-dir: pip tidak simpan cache download di dalam container
# Mengurangi ukuran image secara nyata
RUN pip install --no-cache-dir -r requirements.txt

# Baru di sini seluruh kode disalin
# Layer ini selalu cache miss saat kode berubah — tapi ringan (ratusan kB)
COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4. Membaca docker history

Perintah `docker history <image>` menampilkan semua layer beserta ukurannya. Ini adalah alat diagnostik utama untuk memahami distribusi ukuran image.

Layer	Instruksi	Ukuran	Yang perlu diperhatikan
Base OS	python:3.11-slim layers	~141 MB	Dari registry, tidak bisa diubah
Layer 1	WORKDIR /app	8.19 kB	Sangat ringan
Layer 2	COPY requirements.txt .	12.3 kB	Ringan, jarang berubah

Layer 3 ■	RUN pip install ...	390 MB	MAHAL — harus selalu di-cache
Layer 4	COPY . .	446 kB	Boleh cache miss, ringan dan cepat
Layer 5	EXPOSE + CMD	0 B	Hanya metadata, tidak ada ukuran

Dari hasil eksperimen hari ini: setelah mengedit kode Python dan build ulang, layer pip install tetap menampilkan '25 hours ago' — artinya Docker tidak menyentuhnya sama sekali. Hanya COPY . . yang diulang dalam hitungan detik.

5. Poin koreksi — arah efek domino

Ini adalah poin yang salah dijawab hari ini dan perlu dipahami dengan benar.

■ Kesalahan yang terjadi

Pertanyaan: dalam Dockerfile ini, kamu edit config.json — layer mana yang cache miss?

FROM → WORKDIR → COPY requirements.txt → RUN pip install → COPY config.json → COPY . . → CMD

Jawaban yang salah: 'layer sebelum COPY config.json yang miss'

Ini terbalik. Layer SEBELUM titik perubahan justru tetap di-cache.

■ Jawaban yang benar

Cache HIT (tidak berubah): FROM, WORKDIR, COPY requirements.txt, RUN pip install

Cache MISS (diulang): COPY config.json, COPY . ., CMD

Aturan fundamentalnya: layer yang berubah = cache miss, PLUS semua layer SESUDAHNYA.

Layer sebelumnya tidak terpengaruh sama sekali — mereka sudah di-cache sebelum titik perubahan.

Cara mudah mengingatnya: bayangkan Dockerfile sebagai antrian. Docker memproses dari atas ke bawah. Begitu ada satu langkah yang berubah, semua langkah sesudahnya harus diulang karena hasilnya mungkin berbeda. Langkah sebelumnya sudah selesai dan tidak perlu disentuh.

6. Koneksi ke materi sebelumnya

Layer caching adalah konsekuensi langsung dari layer system yang dipelajari di W09D02. Setiap layer adalah read-only snapshot. Docker tidak 'memodifikasi' layer yang sudah ada — ia hanya memutuskan apakah perlu membuat layer baru atau mengambil yang sudah ada dari cache. Karena layer bersifat immutable, jika satu layer berubah maka semua layer turunannya harus dibuat ulang dari nol — tidak bisa 'patch sebagian'.

Ini juga berkaitan dengan konsep container layer dari W09D01: image terdiri dari read-only layers, container menambahkan writable layer di atas. Layer caching hanya berlaku untuk image layers (saat

docker build) — bukan container layer yang memang ephemeral.

7. Perintah referensi

Perintah	Fungsi
<code>docker build -t name:tag .</code>	Build image dari Dockerfile di direktori saat ini
<code>docker history image:tag</code>	Lihat semua layer beserta ukuran dan instruksi pembuatnya
<code>docker build --no-cache -t name:tag .</code>	Paksa build ulang seluruh layer, abaikan semua cache
<code>pip install --no-cache-dir -r req.txt</code>	Install tanpa simpan cache pip di dalam container — kurangi ukuran image